Course Name:	Competitive Programming Lab	Semester:	IV
Date of Performance:	02/ 03/ 2026	DIV/ Batch No:	B2
Student Name:	Ashwera Hasan	Roll No:	16010124107

Experiment No: 4

Title: Sliding Window Techniques

Aim of the Experiment: To understand and implement Sliding Window Techniques for a given problem

Objective of the Experiment:

After completing this experiment, the student will be able to:

- Understand the concept and working of the Sliding Window technique.
- Identify suitable problem scenarios for applying the Sliding Window approach.
- Implement Sliding Window algorithms efficiently using **C++**.
- Analyze the time and space complexity of the implemented C++ solution.
- Enhance algorithmic problem-solving skills through optimized C++ programs.

COs to be achieved:

CO 2: Analyse and optimize algorithms using amortized analysis and bit manipulation, equipping them to tackle complex computational problems.

Theory:

The Sliding Window technique is an efficient algorithmic approach used to solve problems involving contiguous subarrays or substrings. Instead of repeatedly recalculating values for every possible window, this technique maintains a window of elements and slides it across the data structure (array or string), updating the result incrementally. This significantly reduces time complexity compared to brute-force methods.

A sliding window is defined by two pointers, usually referred to as the **left** and **right** boundaries. As the right pointer moves forward, new elements enter the window, and computations are updated. Depending on the problem, the left pointer may also move forward to shrink the window and maintain certain constraints such as window size or validity conditions.

There are two main types of Sliding Window techniques:

1. **Fixed-size Sliding Window** – where the window size remains constant (e.g., finding the maximum sum of subarrays of size *k*).
2. **Variable-size Sliding Window** – where the window size changes dynamically based on problem constraints (e.g., longest substring without repeating characters).

In C++, Sliding Window algorithms are efficiently implemented using loops, pointers, and data structures such as arrays, strings, hash maps (`unordered_map`), or sets. This approach optimizes performance by reducing nested loops and lowering the overall time complexity from O(n2)O(n^2)O(n2) to O(n)O(n)O(n) in many cases.

Thus, the Sliding Window technique is widely used in solving real-world problems related to arrays, strings, and sequences, making it an essential concept for competitive programming and efficient algorithm design.

Working Mechanism

A typically Sliding Window technique follows these steps:

- Use two pointers (**start** and **end**) to define a window of contiguous elements.
- Initialize both pointers at the beginning of the array or string.
- Move the **end pointer** forward to expand the window by including new elements.
- Update the required computation (sum, count, frequency, etc.) incrementally.
- When the window size reaches a limit or a condition is violated, move the **start pointer** forward to shrink the window.
- Remove the effect of elements leaving the window while shrinking.
- Continue expanding and shrinking the window until all elements are processed.
- Achieve efficient performance by avoiding repeated recalculations, often resulting in **O(n)** time complexity.

Advantages

- Reduces time complexity compared to brute-force approaches.
- Efficient for problems involving **contiguous subarrays or substrings**.
- Avoids redundant calculations by reusing previous results.

- Simple to implement using two pointers in **C++**.
- Improves performance, often achieving **O(n)** time complexity.
- Widely applicable in array and string processing problems.

Limitations

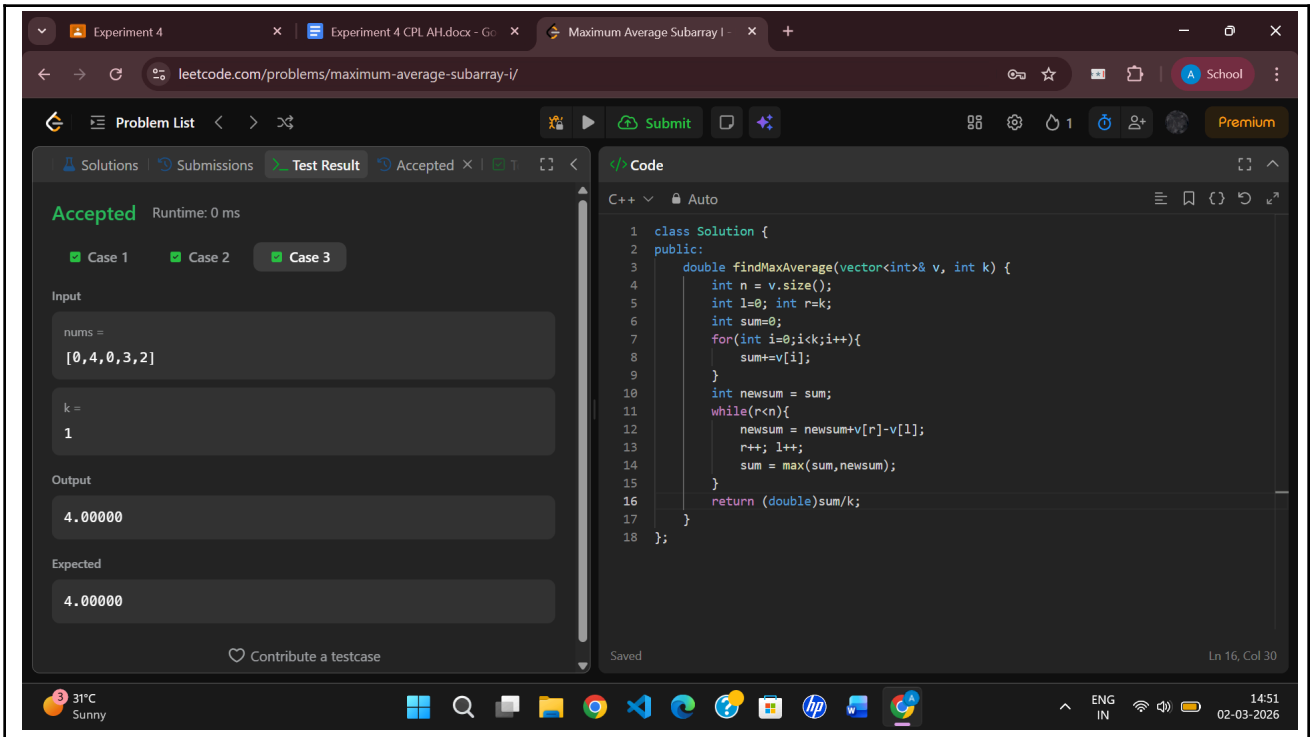
- Applicable only to problems involving **continuous (contiguous) data**.
- Not suitable for problems requiring non-contiguous element selection.
- Can be tricky to design for **variable-size window** problems.
- Requires careful handling of window boundaries and conditions.
- May need extra data structures (maps/sets), increasing space usage.

Problem Statements:

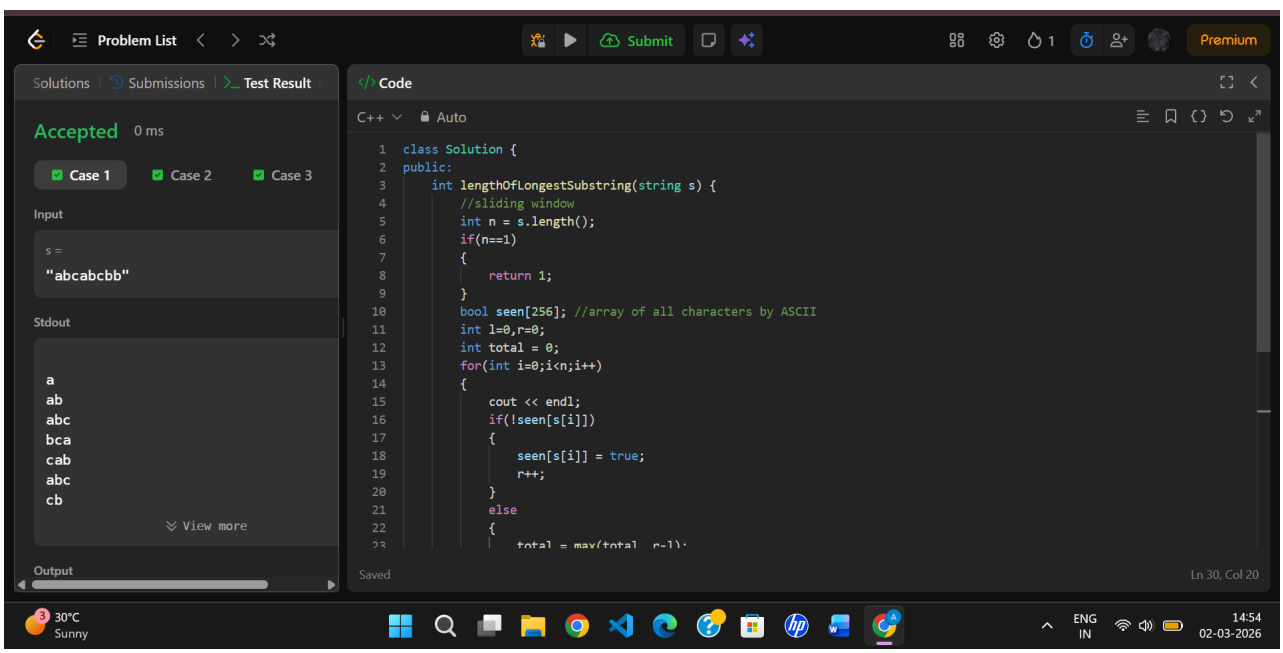
Difficulty (as per lab)	Problem	LeetCode mapping
Easy	0/1 Knapsack (small constraints)	LC 643 – Maximum Average Subarray I
Medium	Matrix chain multiplication	LC 3 – Longest Substring Without Repeating Characters
Difficult	Edit distance	LC 76 – Minimum Window Substring

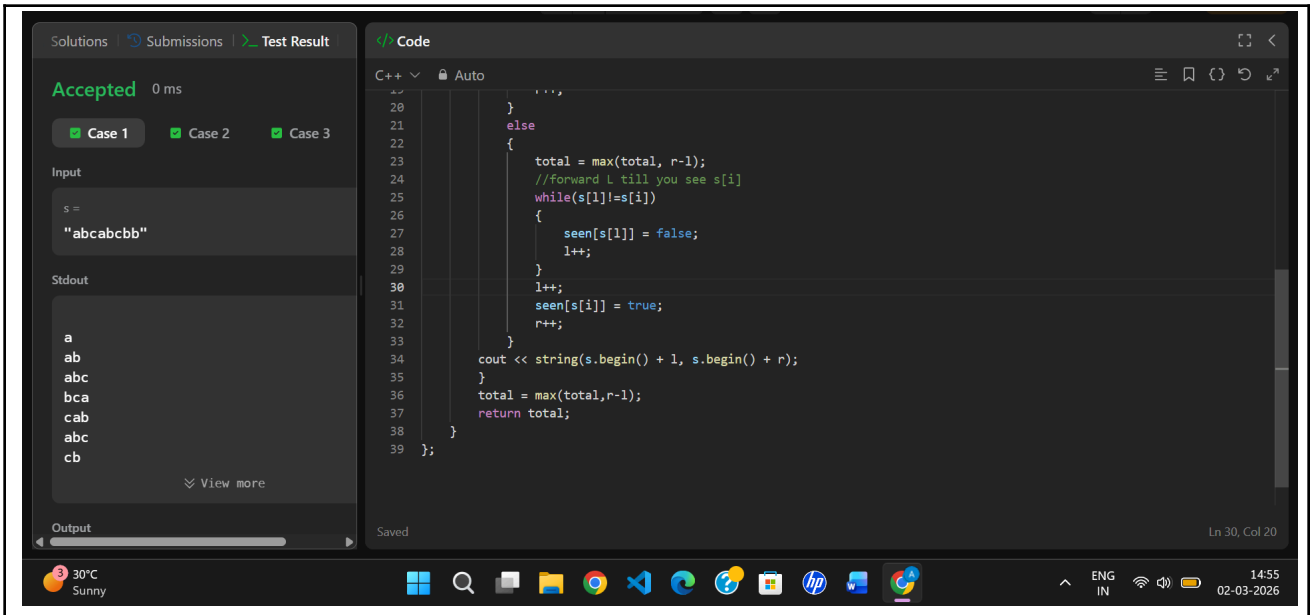
Code :

Easy:

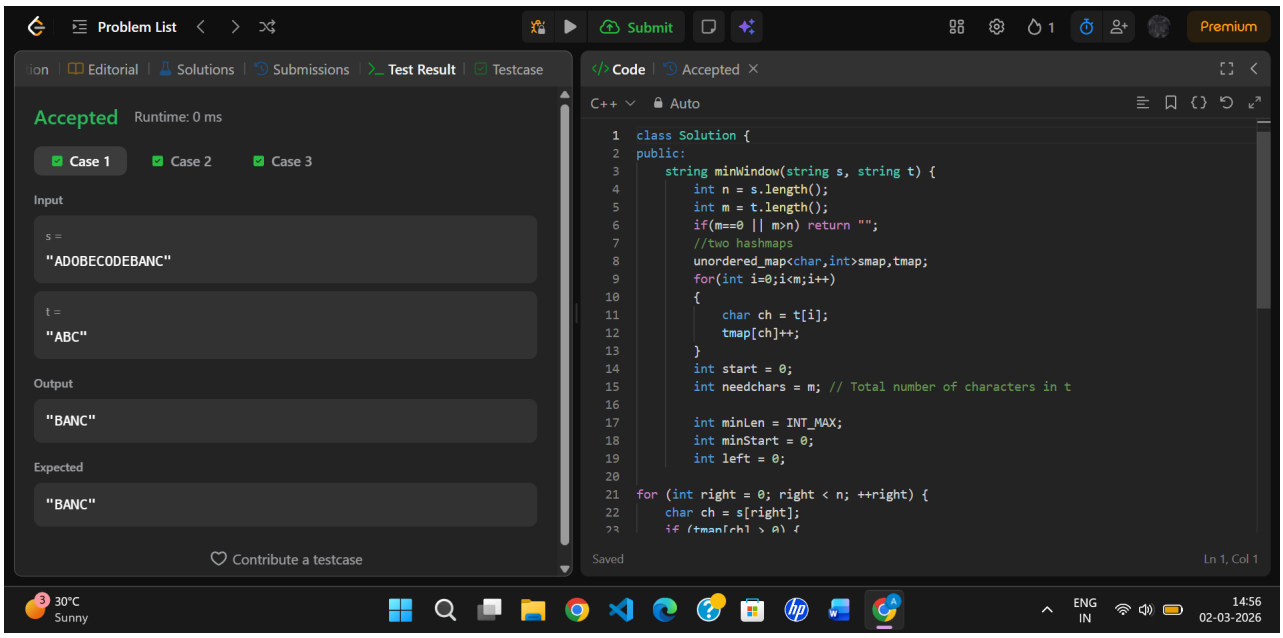

Experiment 4
Experiment 4 CPL AH.docx - Go
Maximum Average Subarray I -
leetcode.com/problems/maximum-average-subarray-i/
Problem List
Submit
Premium
Solutions
Submissions
Test Result
Accepted
Code
Accepted Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[0,4,0,3,2]
k =
1
Output
4.00000
Expected
4.00000
Contribute a testcase
C++
Auto
class Solution {
public:
double findMaxAverage(vector<int>& v, int k) {
int n = v.size();
int l=0; int r=k;
int sum=0;
for(int i=0;i<k;i++){
sum+=v[i];
}
int newsum = sum;
while(r<n){
newsum = newsum+v[r]-v[l];
r++; l++;
sum = max(sum,newsum);
}
return (double)sum/k;
}
};
Saved
Ln 16, Col 30
31°C
Sunny
ENG
IN
14:51
02-03-2026

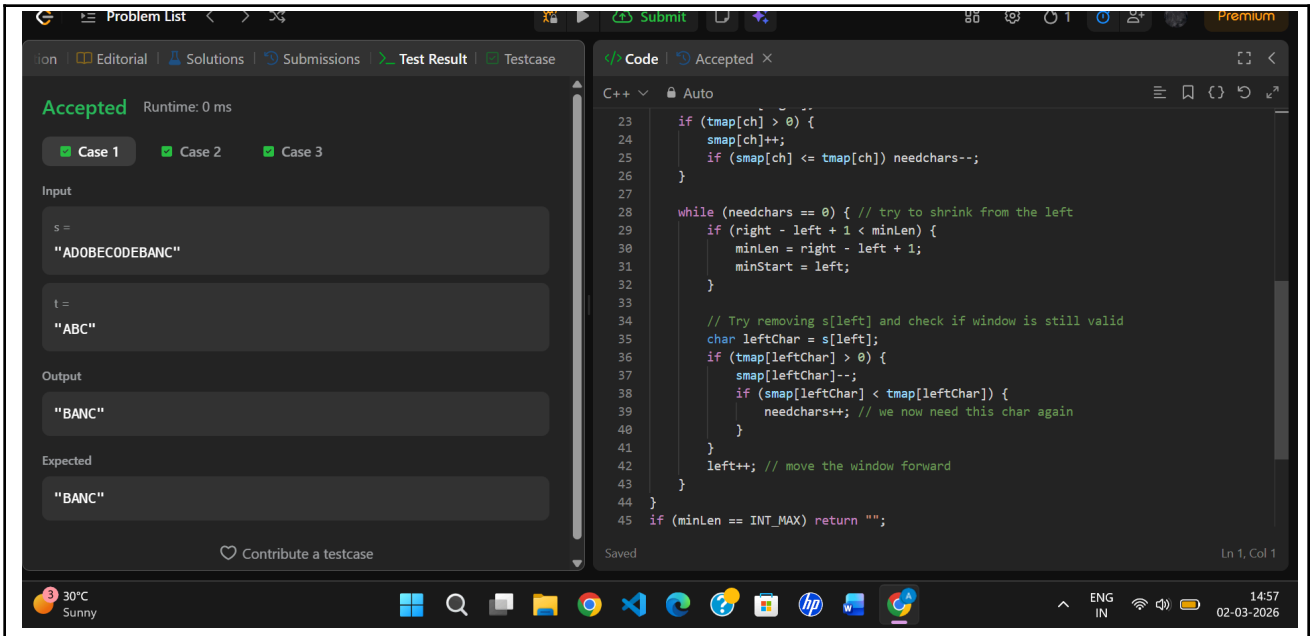
Medium:

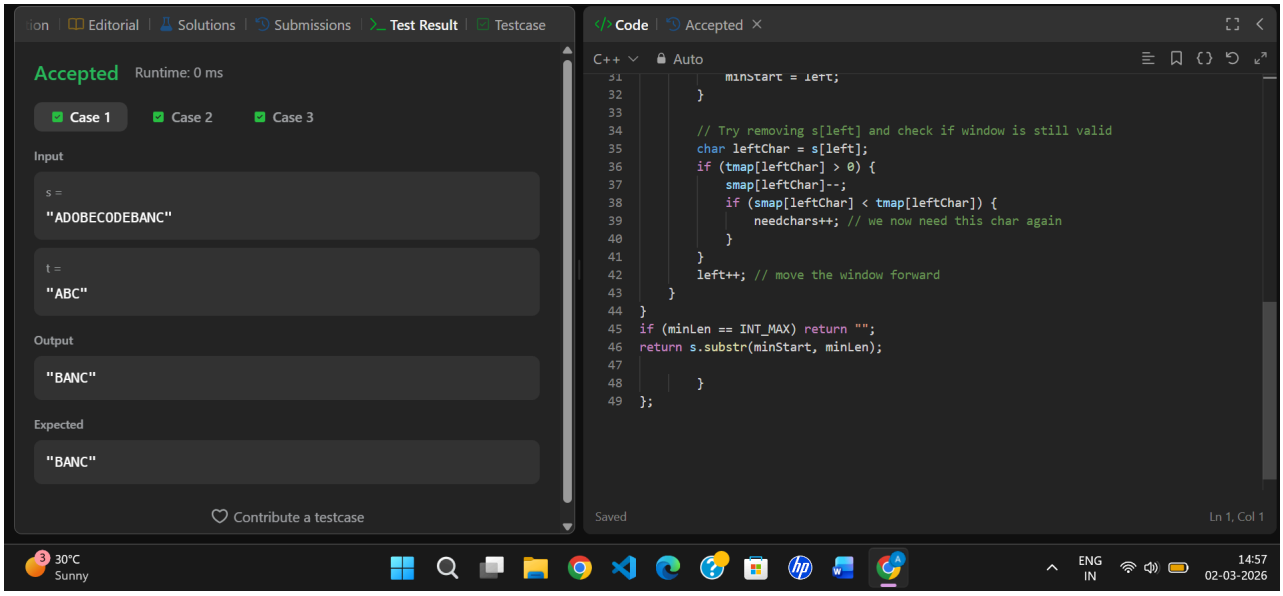

Problem List
Submit
Premium
Solutions
Submissions
Test Result
Code
Accepted 0 ms
Case 1
Case 2
Case 3
Input
s =
"abcabcbb"
Stdout
a
ab
abc
bca
cab
abc
cb
View more
Output
C++
Auto
class Solution {
public:
int lengthOfLongestSubstring(string s) {
//sliding window
int n = s.length();
if(n==1)
{
return 1;
}
bool seen[256]; //array of all characters by ASCII
int l=0,r=0;
int total = 0;
for(int i=0;i<n;i++)
{
cout << endl;
if(!seen[s[i]])
{
seen[s[i]] = true;
r++;
}
else
{
Saved
Ln 30, Col 20
30°C
Sunny
ENG
IN
14:54
02-03-2026


Solutions | Submissions | Test Result
Accepted 0 ms
Case 1 Case 2 Case 3
Input
s =
"abcabcbb"
Stdout
a
ab
abc
bca
cab
abc
cb
View more
Output
Code
C++ Auto
}
else
{
total = max(total, r-l);
//forward L till you see s[i]
while(s[l]!=s[i])
{
seen[s[l]] = false;
l++;
}
l++;
seen[s[i]] = true;
r++;
}
cout << string(s.begin() + l, s.begin() + r);
}
total = max(total,r-l);
return total;
}
};
Saved
Ln 30, Col 20
30°C Sunny
ENG IN
14:55
02-03-2026

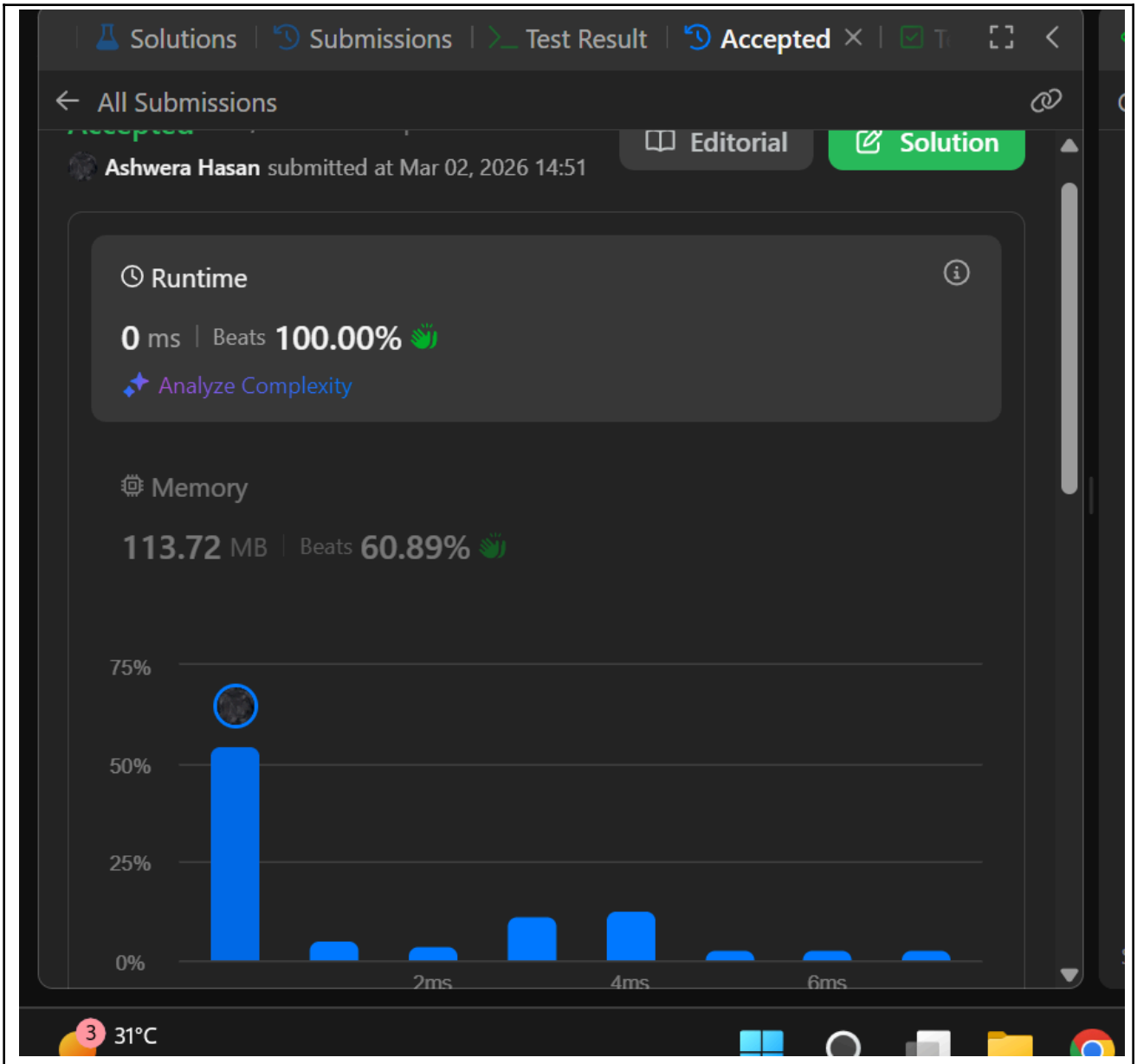
Hard:


Problem List
Submit
Premium
Editorial | Solutions | Submissions | Test Result | Testcase
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
Input
s =
"ADOBECODEBANC"
t =
"ABC"
Output
"BANC"
Expected
"BANC"
Contribute a testcase
Code | Accepted
C++ Auto
class Solution {
public:
string minWindow(string s, string t) {
int n = s.length();
int m = t.length();
if(m==0 || m>n) return "";
//two hashmaps
unordered_map<char,int>smap,tmap;
for(int i=0;i<m;i++)
{
char ch = t[i];
tmap[ch]++;
}
int start = 0;
int needchars = m; // Total number of characters in t
int minLen = INT_MAX;
int minStart = 0;
int left = 0;
for (int right = 0; right < n; ++right) {
char ch = s[right];
Saved
Ln 1, Col 1
30°C Sunny
ENG IN
14:56
02-03-2026

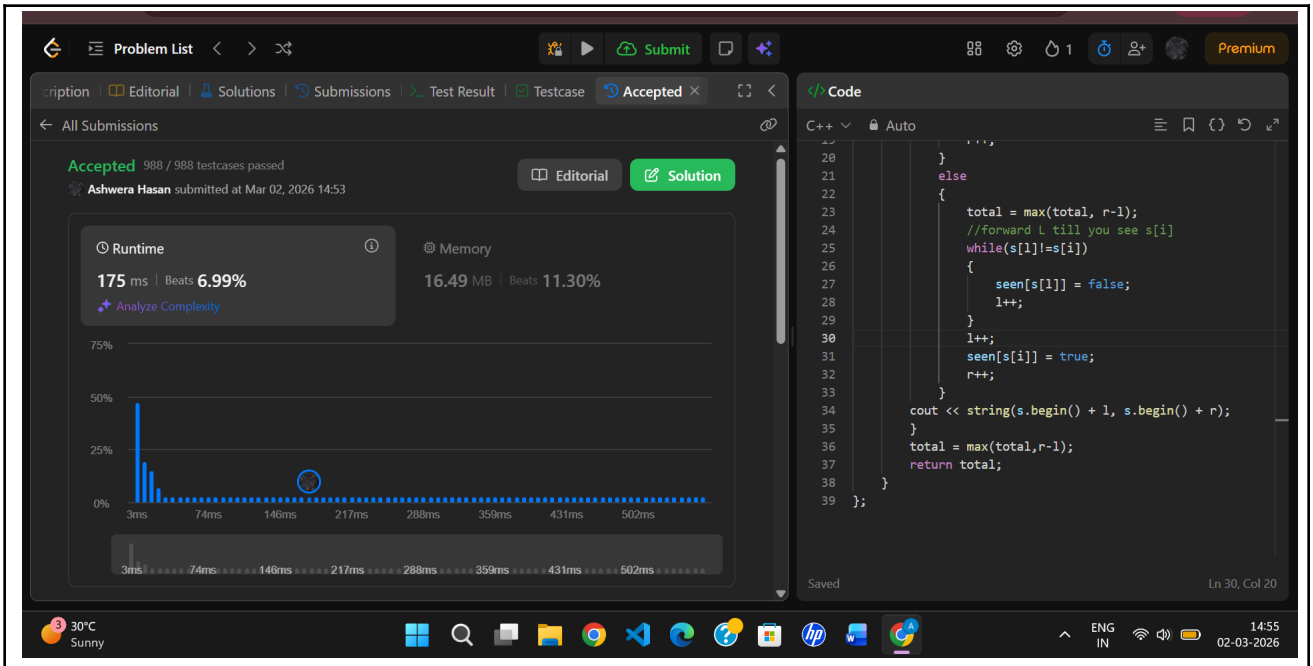

Problem List
Submit
Premium
Editorial
Solutions
Submissions
Test Result
Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
s =
"ADOBECODEBANC"
t =
"ABC"
Output
"BANC"
Expected
"BANC"
Contribute a testcase
Code
Accepted
C++
Auto
if (tmap[ch] > 0) {
smap[ch]++;
if (smap[ch] <= tmap[ch]) needchars--;
}
while (needchars == 0) { // try to shrink from the left
if (right - left + 1 < minLen) {
minLen = right - left + 1;
minStart = left;
}
// Try removing s[left] and check if window is still valid
char leftChar = s[left];
if (tmap[leftChar] > 0) {
smap[leftChar]--;
if (smap[leftChar] < tmap[leftChar]) {
needchars++; // we now need this char again
}
}
left++; // move the window forward
}
}
if (minLen == INT_MAX) return "";
Saved
Ln 1, Col 1
30°C
Sunny
ENG
IN
14:57
02-03-2026


Editorial
Solutions
Submissions
Test Result
Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
s =
"ADOBECODEBANC"
t =
"ABC"
Output
"BANC"
Expected
"BANC"
Contribute a testcase
Code
Accepted
C++
Auto
minStart = left;
}
// Try removing s[left] and check if window is still valid
char leftChar = s[left];
if (tmap[leftChar] > 0) {
smap[leftChar]--;
if (smap[leftChar] < tmap[leftChar]) {
needchars++; // we now need this char again
}
}
left++; // move the window forward
}
}
if (minLen == INT_MAX) return "";
return s.substr(minStart, minLen);
}
};
Saved
Ln 1, Col 1
30°C
Sunny
ENG
IN
14:57
02-03-2026

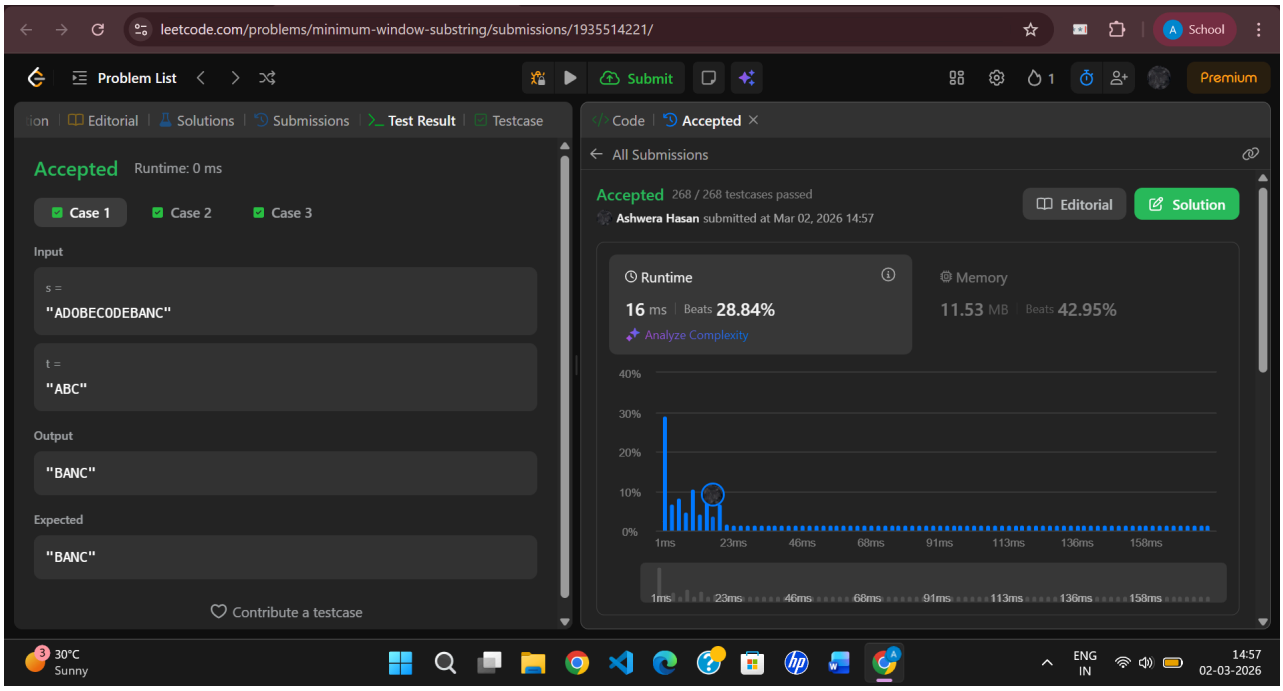
Output:
Easy:


Solutions
Submissions
Test Result
Accepted
All Submissions
Editorial
Solution
Ashwera Hasan submitted at Mar 02, 2026 14:51
Runtime
0 ms | Beats 100.00%
Analyze Complexity
Memory
113.72 MB | Beats 60.89%
75%
50%
25%
0%
2ms
4ms
6ms
31°C

Medium:



Hard:



Post Lab Subjective/Objective type Questions:

1. Explain how Sliding Window ensures **O(n)** time complexity even though two pointers are used.
   A sliding window ensures an O(n) time complexity despite having two pointers since one pointer only ever moves across one element at most once.
   This ensures that we avoid double checking of the same numbers twice, reducing the complexity to O(n).
2. Given an array containing negative numbers, does the Sliding Window technique always work? Explain your answer.
   A sliding window technique will not always work in an array containing negative numbers. Some modification will be required to make the array work like a regular sliding window. For example, with the introduction of negative numbers in sliding windows, it is not always guaranteed that increasing one pointer will increase the sum and hence we might have to restart pointers upon encountering such a situation.

Conclusion:

The sliding window is a powerful technique that can be used to reduce the time complexity of seemingly $O(n^2)$ problems, allowing us to solve them in O(n) time.

Signature of faculty in-charge with Date: